

COMMUNICATION LABORATORY

EXPERIMENT 1: PN SEQUENCES



NAME: N DEEKSHITHA

ROLL NO: 24EC10105

GROUP 14 (Tuesday Batch)

Design and Simulation Laboratory-Communication Systems

Department of Electronics & Electrical Communication Engineering

Indian Institute of Technology Kharagpur

TITLE: Design of Pseudo-Noise (PN) sequences using Linear Feedback Shift Register (LFSR) and applications of PN sequences.

PROBLEM STATEMENT: Design a 5-bit pseudo noise (PN) sequence generator using a linear feedback shift register (LFSR). In particular,

1. Demonstrate a simple construction of a PN sequence.
 2. Now robustify the design by avoiding the 'lockout' condition (where all register states are set to '0').
 3. Find the correlation function of the generated PN sequence by taking suitable shifted versions of the sequence.
 4. Using simple arithmetic (modulo-2) operations, construct, generate and demonstrate a Gold code sequence.
-

KEY COMPONENTS: IC7418(AND gate), IC7432(OR gate), IC7486(XOR gate), IC7495(4-bit shift register), IC74164 (8-bit shift register), digital oscilloscope, function generator, breadboard, connecting wires, power supply.

BRIEF THEORY & BACKGROUND:

A. Pseudo-Noise (PN) Sequence

The experiment's objective is to generate a pseudo-noise (PN) sequence through the utilization of a linear shift feedback register (LFSR) [1]. An LFSR functions as a generator of random number sequences, serving as a cipher within cryptography. PN sequences find frequent application in digital communication systems, oftentimes also approximating white noise. Often denoted as Pseudo-Random Binary Sequences (PRBS), PN sequences possess a 'pseudo' nature, indicating that their randomness is not absolute. Instead, they exhibit periodicity with the potential for a substantial period, showcasing characteristics of a random white sequence within this interval.

B. Linear Feedback Shift Register (LFSR)

PN sequences arise from the assembly of a fixed number of flip-flops connected serially, coupled with combinational logic, typically in the form of XOR gates. The contents of the registers shift one bit to the right with each rising edge of the clock pulse. Simultaneously, the logic circuit computes a Boolean function based on the flip-flop states. This outcome loops back as input to the initial flip-flop, thus preventing the shift register from emptying. The resulting PN sequence is contingent upon factors like the number of employed flip-flops, the feedback circuit's logic, and the initial state.

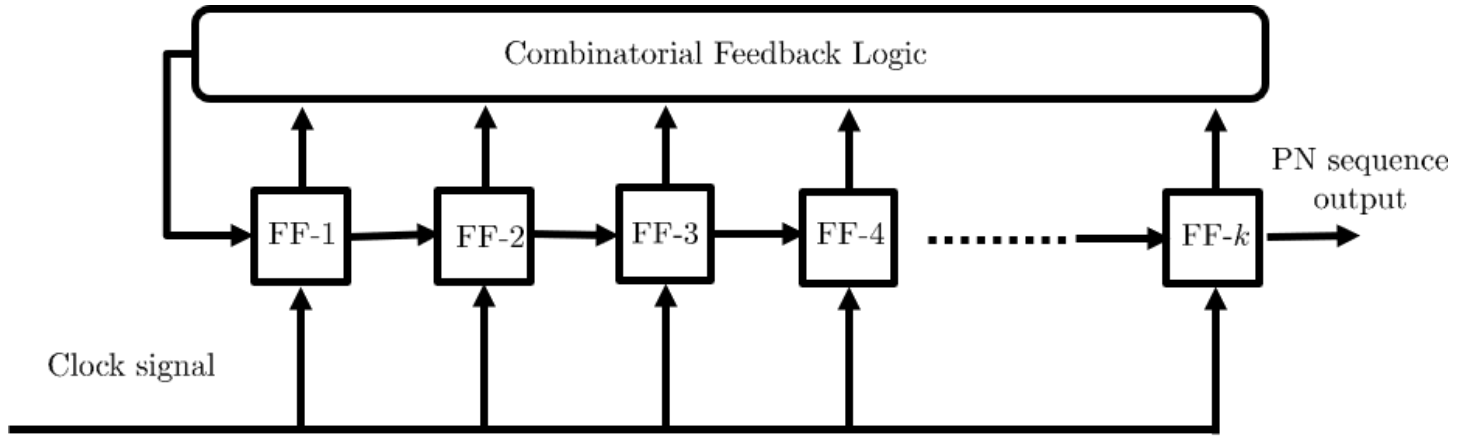


Figure 1: Block diagram of a typical PN sequence generator using an LFSR.

C. Primitive Polynomial

The feedback taps of an LFSR are determined by its characteristic polynomial over the finite field $GF(2)$. When the characteristic polynomial is primitive, the LFSR generates a maximum-length (m-) sequence having a period: $N=2^n-1$ where n is the number of register stages. Primitive polynomials ensure that every non-zero register state is visited exactly once before the sequence repeats. For the present experiment, the feedback polynomial x^5+x^3+1 is used to generate a maximal-length PN sequence.

D. Lockout Condition

The generated sequence depends on the feedback polynomial and the initial state of the register. The all-zero state is excluded because it results in the "lockout" condition. Since LFSRs use XOR gates for feedback, if all bits are zero, the XOR operation will always produce a zero, preventing the register from changing its state. The register continuously remains in the zero state unless acted upon by external inputs. Therefore, the LFSR must always be initialized with a non-zero seed or provided with suitable lockout protection circuitry to detect the all-zero state and force the register to a non-zero state.

E. Autocorrelation

One of the most important characteristics of PN sequences is their excellent autocorrelation property. The autocorrelation function $R_{xx}(k)$ of a sequence $x(i)$ of length N is defined as:

$$\frac{1}{N} \sum_{i=0}^{N-1} x(i)x(i+k)$$

The autocorrelation function exhibits a sharp peak (maxima) when the shift k is zero or multiples of the sequence period. For all other shifts, the correlation value remains uniformly low. This property makes PN sequences suitable for synchronization, ranging, and spread spectrum communication systems.

Furthermore, according to the **Wiener-Khinchin Theorem**, the autocorrelation of a signal is mathematically equivalent to the inverse Fourier transform of its Power Spectral Density (PSD). The PSD itself is the magnitude squared of the Fourier transform. This property allows the autocorrelation to be practically computed on hardware oscilloscopes using built-in Fast Fourier Transform (FFT) math functions.

F. Gold Sequence

Gold sequences are a special type of binary sequence generated by performing a modulo-2 addition (XOR) of two suitable PN sequences of equal length, identical periods, and a common generating clock. The resulting Gold sequence produces a larger family of sequences with excellent cross-correlation properties, which is highly desirable for applications such as spreading codes in Code Division Multiple Access (CDMA) systems.

G. Properties of Maximum-Length PN Sequences

The properties that make a pseudorandom sequence appear statistically random are listed below:

- **Balance Property:** In each period of a maximum length sequence, the number of 1s is always one more than the number of 0s.
- **Run Property:** Among the runs of 1s and 0s in each period of a maximum length sequence, one half the runs of each kind are of length one, one-fourth are of length two, one eighth are of length three, and so on as long as these fractions represent meaningful numbers of runs.
- **Correlation Property:** Autocorrelation has a sharp peak at zero shift and low values elsewhere. Cross-correlation is a measure of similarity between two different sequences.
- **Periodicity Property:** A maximum-length PN sequence generated using an n -stage LFSR repeats after $(2^n - 1)$ bits. For a 5-stage LFSR, the generated sequence has a maximum period of 31 bits before repeating [2], [5].

H. Applications of PN sequence in communication and signal processing systems

PN sequences are extensively used in communication and signal processing systems because of their deterministic generation and favourable correlation properties [3]–[5]. Some important applications include:

- **Spread Spectrum Communication:** Used as spreading codes in Direct Sequence Spread Spectrum (DSSS) systems to improve resistance against interference, jamming, and multipath fading.
- **Code Division Multiple Access (CDMA):** Different users are assigned distinct PN or Gold codes, enabling multiple users to share the same frequency band simultaneously with minimal mutual interference.
- **Synchronization:** The sharp autocorrelation peak of PN sequences enables accurate timing and frame synchronization in digital communication receivers.
- **Channel Estimation:** Known PN sequences are transmitted as training sequences to estimate the characteristics of wireless communication channels.

- **Radar, Sonar and Navigation:** Employed for pulse compression, ranging, and target detection due to their favourable autocorrelation characteristics.
- **Cryptography and Built-In Self-Test (BIST):** Used for data scrambling, stream cipher generation, and pseudorandom test pattern generation in digital integrated circuit testing.

EXPECTED OUTCOMES:

1. Utilizing an 8-bit linear feedback shift register, it becomes feasible to produce a PN sequence with a potential length extending up to 255 bits. Demonstrate a PN sequence using 5-bit shift registers, and comment on the size of the PN sequence generated. Check that the generated sequence adheres to the fundamental PN properties, including the run length, balance, and correlation properties. For the resulting output plot, include both the clock and the generated PN sequence on a common reference.

Note: to avoid the lockout condition, make the input to the first flip flop temporarily floating. This ensures that the initial state of the LFSR is not all-zeros.

2. Lockout conditions can force a PN sequence generator to “lock” itself to the all zero state where all shift registers are at “0” state. Given the combinatorial feedback logic, an LFSR in the all-zero state continues to be in that state unless acted upon by external inputs. Hence, it is pertinent to build accompanying protection to help avoid getting into the all-zero or lockout state. Design the circuit for *lock-out protection*.

3. For the stated problem (generation of a 5-bit PN sequence), the autocorrelation function's peak should exclusively manifest when the shift amounts to either zero or multiples of the generated PN sequence's period. Contrarily, during other instances, the correlation function remains consistently low, showing the minimum value. Consequently, the generated PN sequence should exhibit a favourable characteristic of elevated autocorrelation and diminished cross-correlation. Plot the autocorrelation function.

4. Gold sequences are widely used in spread spectrum communication. Here is a brief background, after which the specific problem follows:

Gold sequences are a special type of binary sequence that are generated by XORing two PN sequences. As we know already, PN sequences, or pseudorandom noise sequences, are binary sequences that exhibit some of the statistical properties of white noise, but are generated deterministically from a shift register with feedback. To generate a Gold sequence, we need to choose two same-length PN sequences with identical periods and a common generating clock. Once the two PN sequences are generated, the final Gold sequence is generated by XORing the PN sequences component-wise. The resulting Gold sequence will also have good autocorrelation properties, which makes it useful for applications such as spreading codes in CDMA systems.

a) You need to design a length $N = 127$ sized Gold sequence.

b) Draw a block diagram of the Gold sequence generation.

c) Show a plot of the Gold sequence with respect to one of the underlying PN sequence.

d) Can you comment on the autocorrelation property of the Gold sequence vis-a-vis the underlying PN sequences (which have a two-valued autocorrelation function, i.e., one “high” value and one low “value”)? *Hint: it is more than 2-valued, i.e., it takes more than 2 values.*

For software parts:

1. Export the DSO data, i.e., time series in .csv format and plot it in MATLAB.
2. Plot the autocorrelation function for the above data and comment on the generated function.
3. In addition, generate a PN sequence with 6-stage Linear Feedback Shift Register and plot its autocorrelation function using the following polynomial:

$$x^6 + x + 1$$

4. Repeat step 3 for the following polynomial:

$$x^6 + x^5 + x^4 + x^3 + x^2 + x + 1$$

5. Comment on the difference in the plots, if any, emphasizing, in particular, on the nature of the polynomials (irreducible and primitive, if any).

Note: *Both hardware and software results should be included in the report. Standard MATLAB functions may be used.*

OBSERVATIONS AND RESULTS:

1. Generation of 5-Bit PN Sequence

Design: A 5-stage LFSR implementing the feedback primitive polynomial x^5+x^3+1 was constructed using an IC74164 (8-bit Serial-In, Parallel-Out shift register) and an IC7486 (Quad 2-input XOR gate). To realize a 5-bit LFSR, the first five outputs (QA through QE) of the IC74164 were utilized. The polynomial dictates that the feedback should be taken from the 3rd and 5th stages. Therefore, outputs QC (Pin 5) and QE (Pin 10) of the IC74164 were connected to the inputs of the XOR gate (Pins 1 and 2 of IC7486). The output of the XOR gate (Pin 3) was fed back into the serial inputs A and B (Pins 1 and 2) of the shift register. A square-wave clock from a function generator was applied to the CP pin (Pin 8) to synchronize register shifting.



Figure 2: Generated 5-bit PN sequence.

Observation: The observed PN sequence exhibited periodic pseudo-random behaviour with uniform transitions. The experimentally observed sequence matched the theoretically expected sequence generated by the feedback polynomial, and adhered to the balance and run-length properties.

Hardware Autocorrelation via Oscilloscope Math: To directly observe the autocorrelation of the PN sequence in real-time without external software, the oscilloscope's built-in math functions were utilized based on the Wiener-Khinchin theorem. By setting the math function to **fftmag(fftmag(CH2)*fftmag(CH2))**, the oscilloscope calculates the Power Spectral Density (PSD) and performs a subsequent Fourier transform (equivalent to the inverse transform for this even function), thereby displaying the autocorrelation plot live on the screen.

2. Lockout Protection

Design: To build accompanying protection against the lockout condition (where all shift registers fall into the "0" state causing the sequence to stagnate), a 2:1 Multiplexer (IC74157) and OR gate logic (IC7432) were implemented in the feedback loop. By taking a logical OR of the register outputs, the OR logic outputs a 0 only when the register is in the 00000 lockout state; otherwise, it outputs a 1. This OR output is connected to the Select (S) input of the 2:1 MUX (Pin 1).

- The I0 (Data 0) input of the MUX (Pin 2) is tied to a High voltage (+5V or Logic 1).
- The I1 (Data 1) input of the MUX (Pin 3) receives the normal XOR feedback ($QC \oplus QE$).

When the lockout state occurs, the Select line is 0, causing the MUX to select I0 and forcefully inject a 1 into the serial input, breaking the lockout. For all other states, the Select line is 1, causing the MUX to select I1 and normal PN sequence generation continues.

Figure 3: Lockout Protection Circuit Schematic

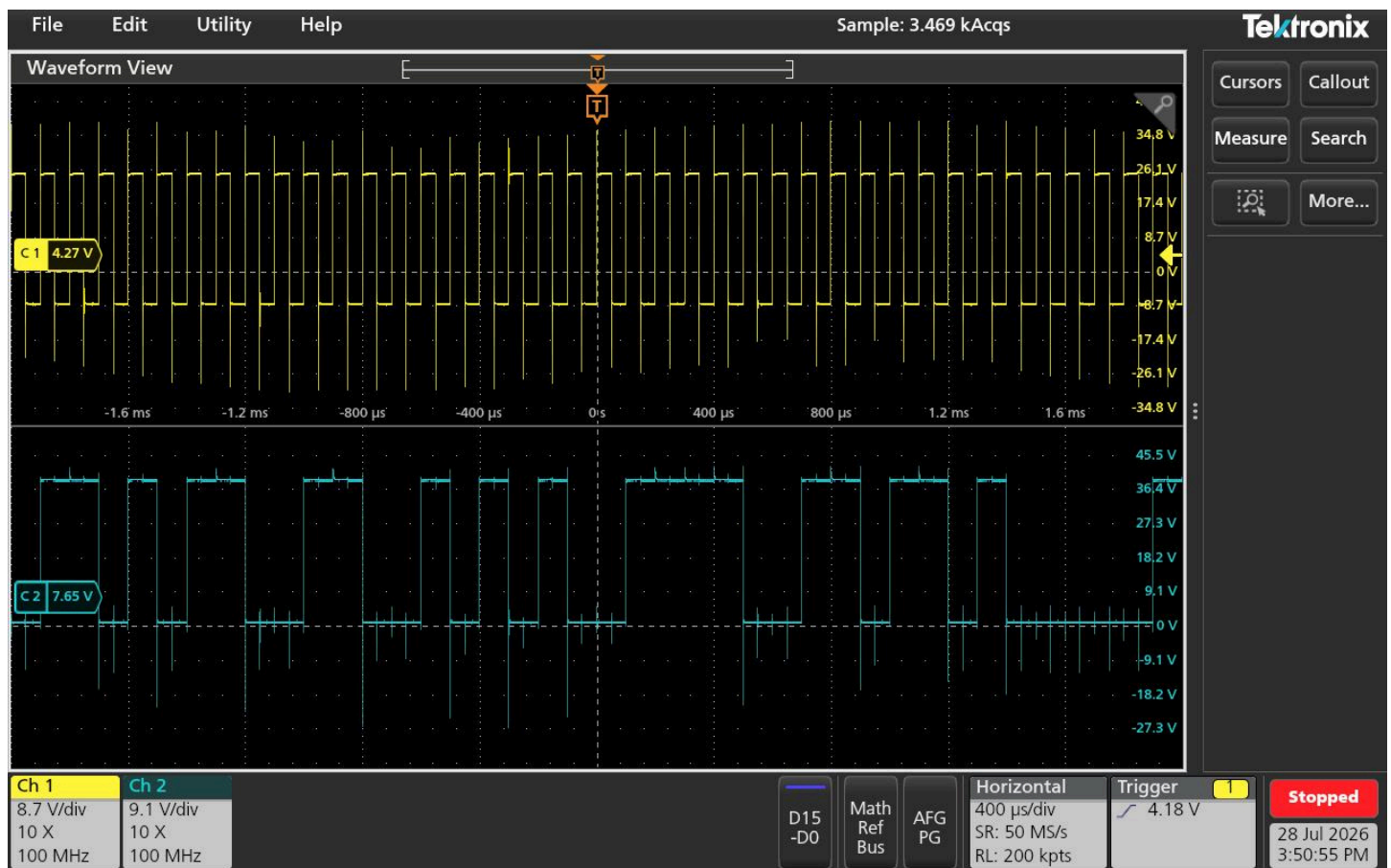


Figure 4: Oscilloscope waveform of PN1 (x^7+x^4+1)

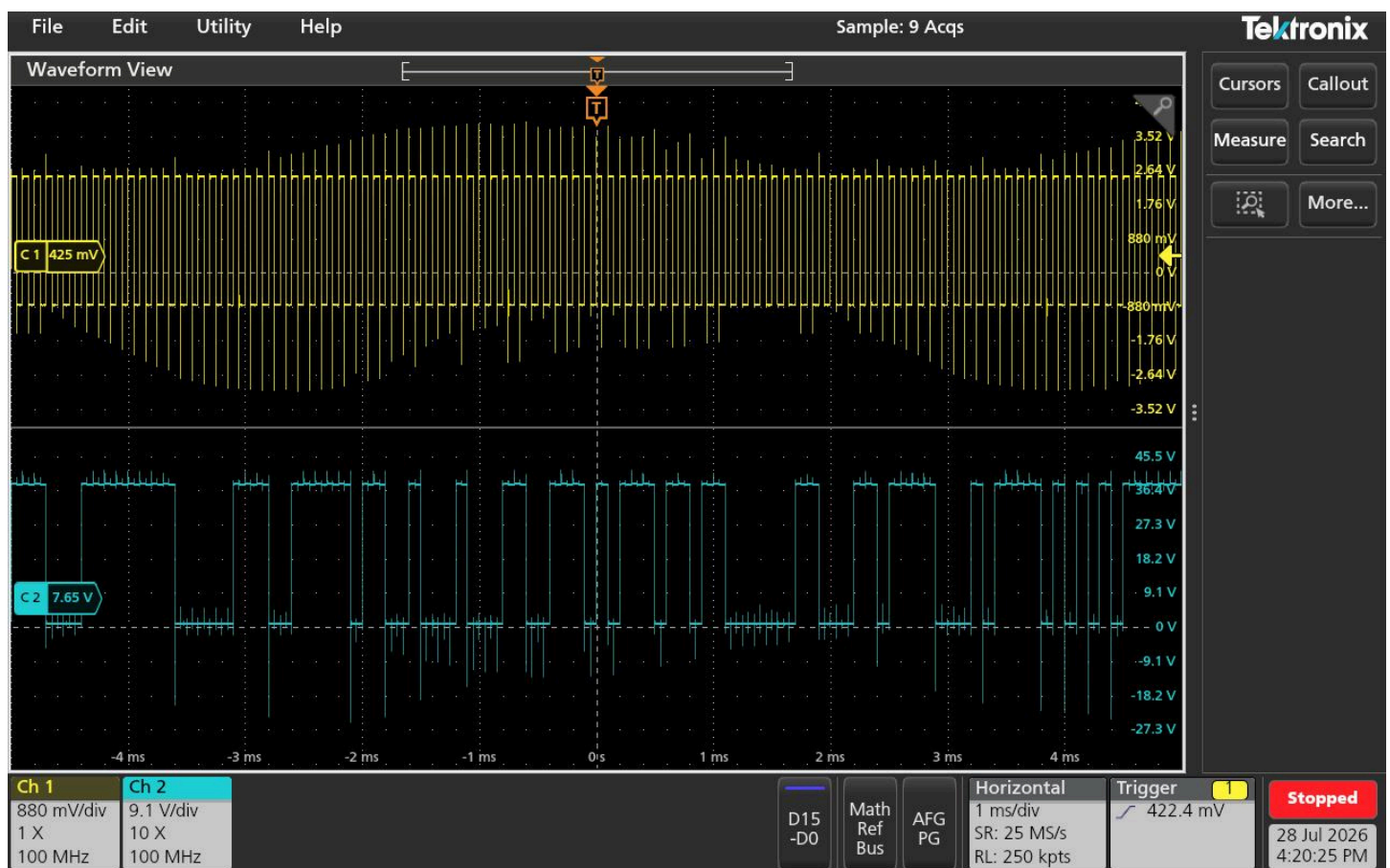


Figure 5: Oscilloscope waveform of PN2 (x^7+x+1)

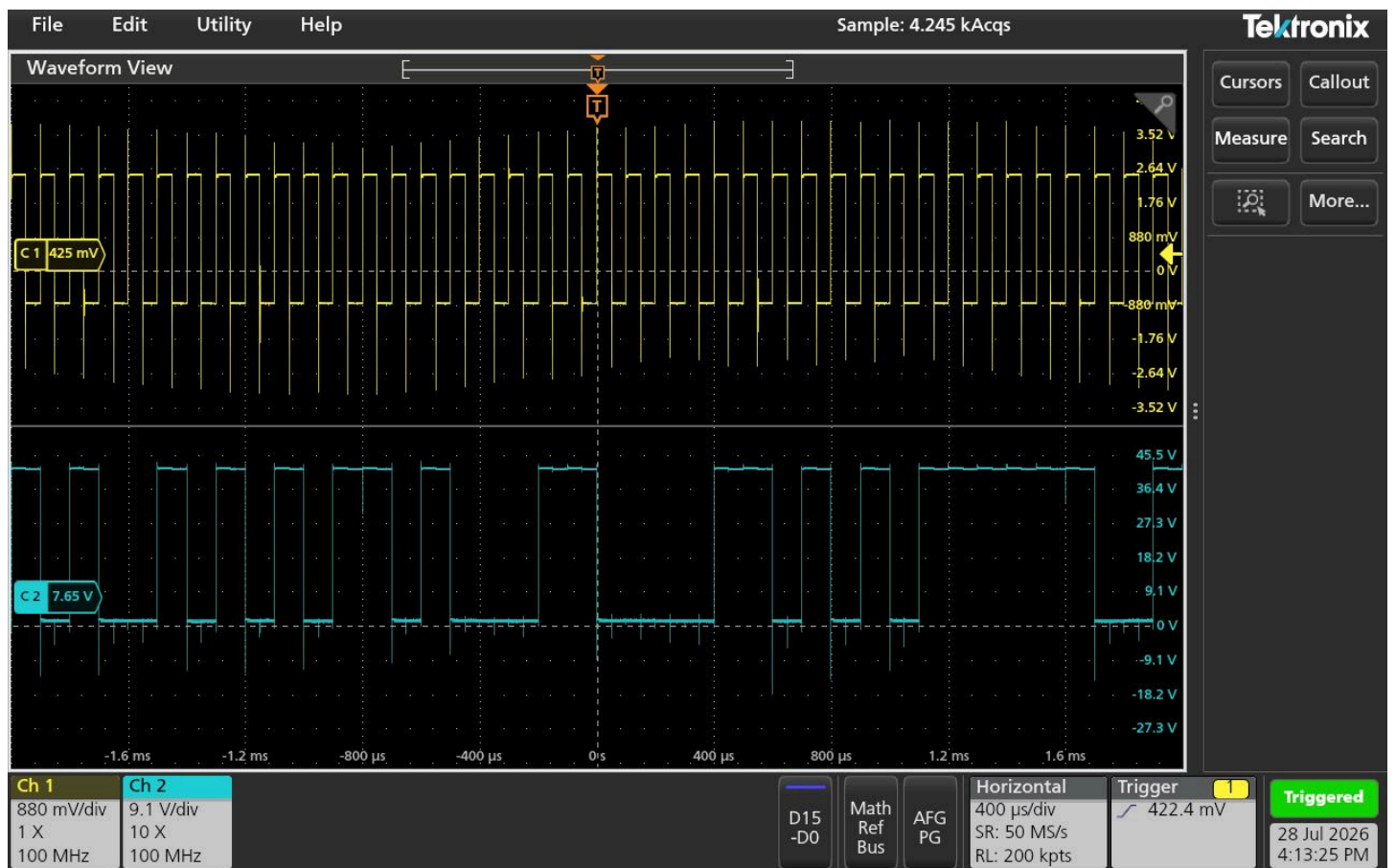


Figure 6: Oscilloscope waveform Gold Sequence (PN1 $\oplus$ PN2)

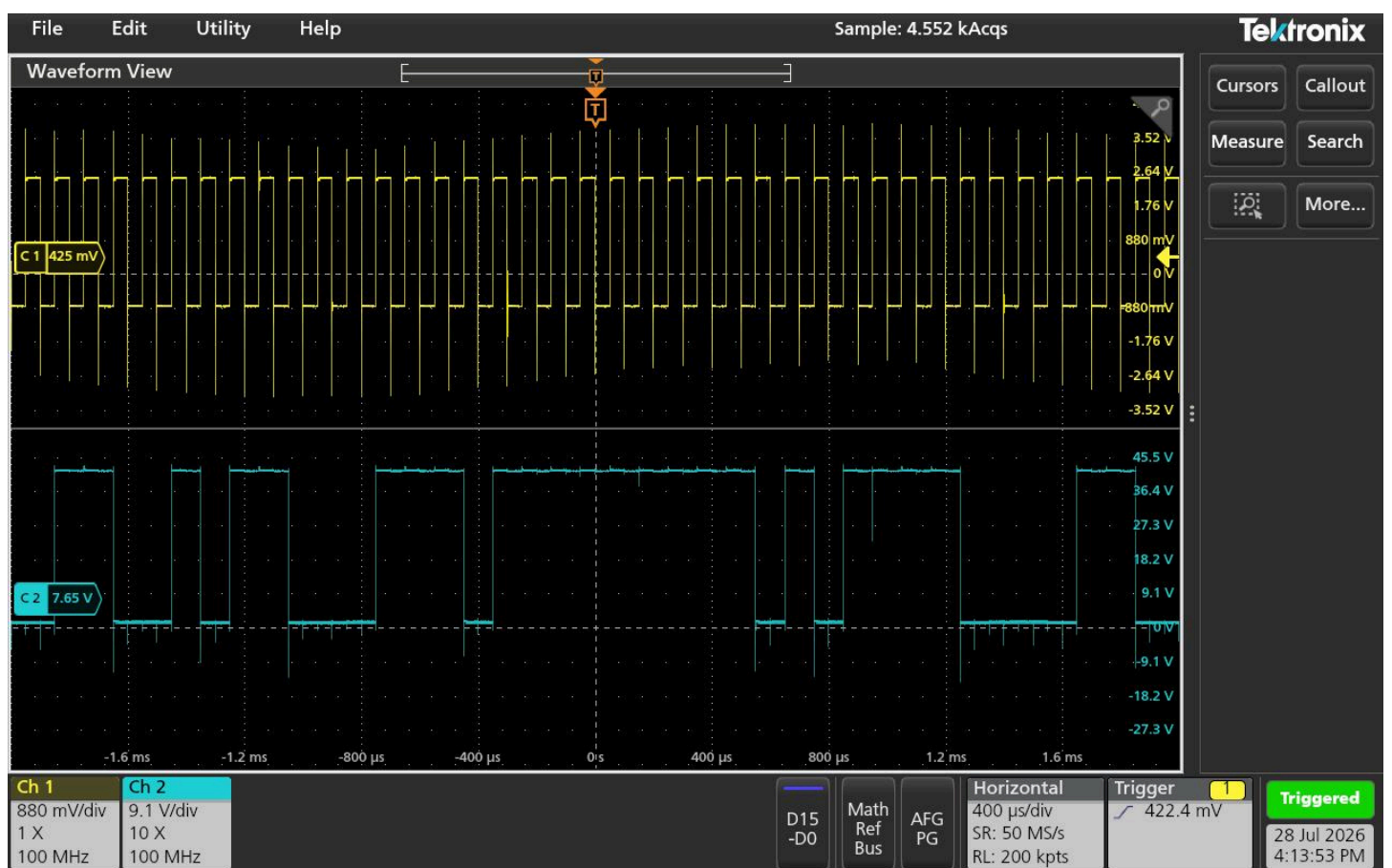


Figure 7: Oscilloscope waveform Gold Sequence (PN1 $\oplus$ PN2)

4. Software Results

Autocorrelation from DSO Data: The raw continuous-time voltage data from the DSO was exported in .csv format and processed in MATLAB. Because the physical signal consists of rectangular voltage pulses rather than abstract discrete points, its theoretical autocorrelation mathematically yields a sequence of triangle functions.

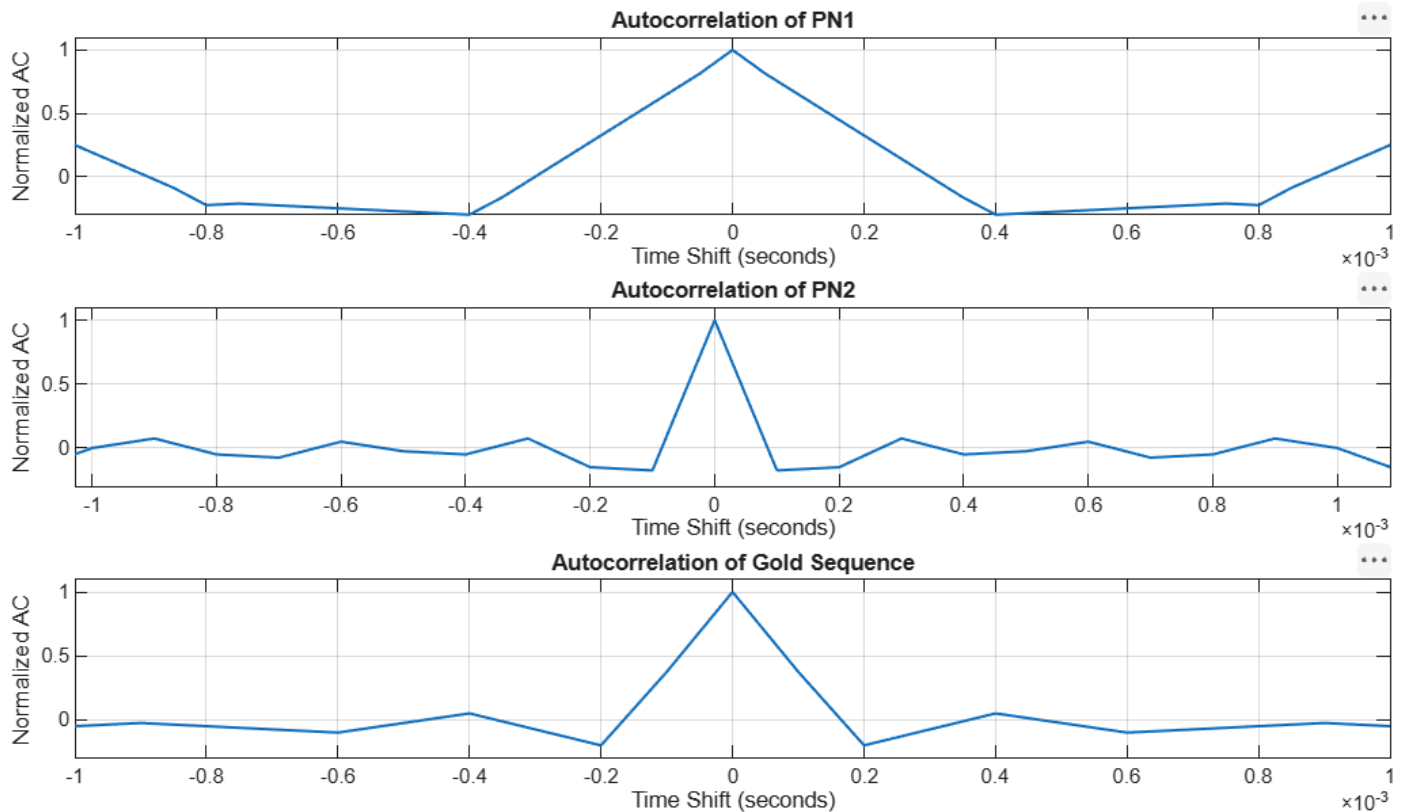


Figure 8: Hardware Autocorrelation Plots of PN1, PN2, and Gold Sequence here

Observation:

- **PN1 and PN2:** The autocorrelation function exhibited sharp triangle peaks at zero-shift (and multiples of the period) and remained uniformly low and flat for all other shifts. This confirms the ideal two-valued autocorrelation property of maximum-length PN sequences.
- **Gold Sequence:** The plot showed the main triangle peaks at multiples of the period, but also exhibited smaller, jagged triangle ripples between the main peaks. This beautifully demonstrates the 4-valued cross-correlation property of Gold sequences, which sacrifice perfect flatness for bounded, predictable cross-correlation across different codes.

Numerical Simulations with 6-stage LFSR: A software simulation of a 6-stage LFSR was written in MATLAB to generate the discrete sequences for two different polynomials. The sequences were generated iteratively using modulo-2 (XOR) feedback, and their discrete autocorrelation was computed using a circular shift method.

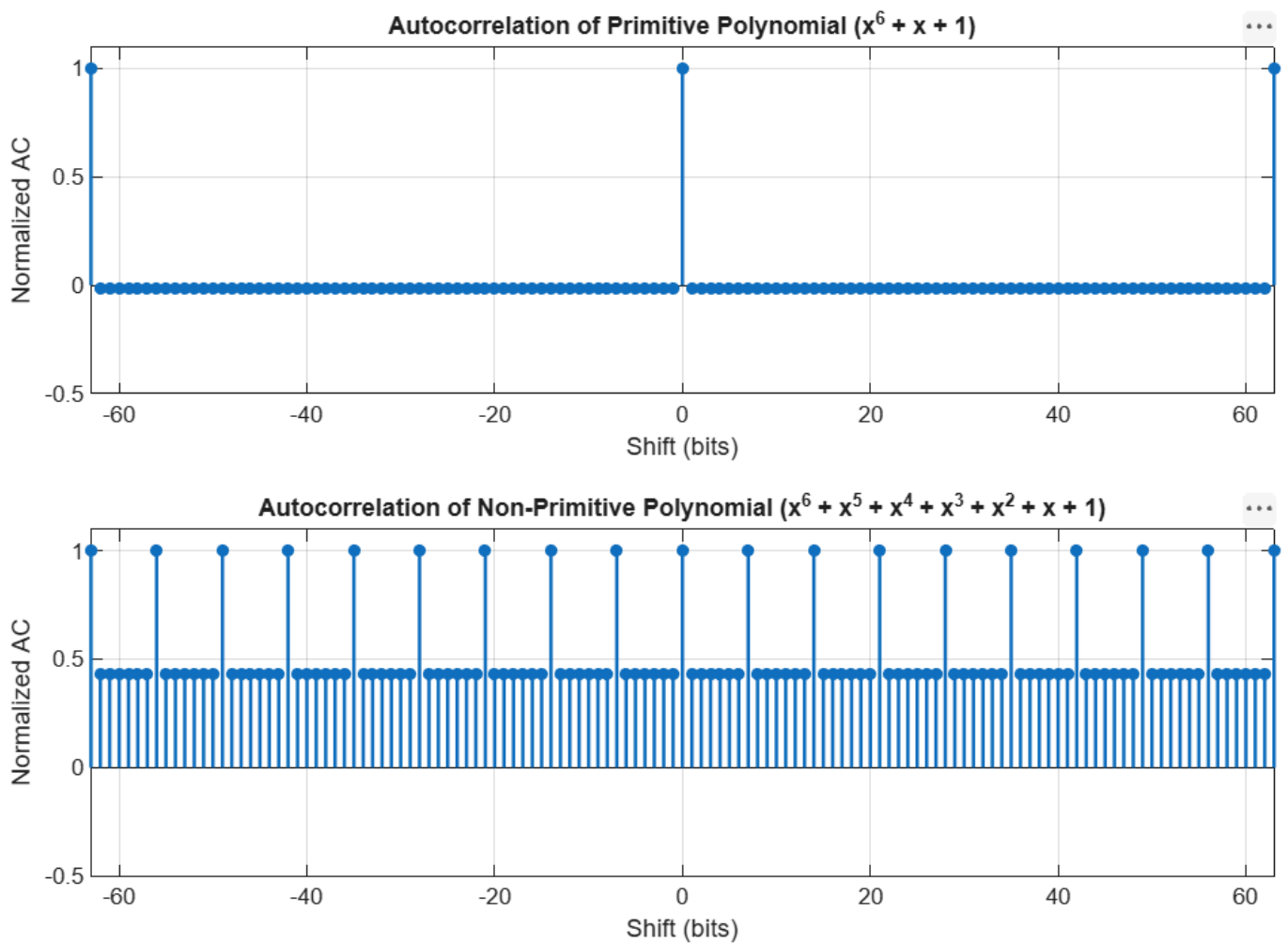


Figure 9: Software Autocorrelation Stems for Primitive vs Non-Primitive Polynomials here

Matlab Code:

```
clear; clc; close all;
files = {'pn1.csv', 'pn2.csv', 'golden sequence.csv'};

plottitles = {'Autocorrelation of PN1', 'Autocorrelation of PN2', 'Autocorrelation of Gold Seq'};
figure(1);

% loop through the hardware data
for i = 1:3

    disp(['loading ' files{i} '...']);

    d = readmatrix(files{i}, 'NumHeaderLines', 14);

    t = d(:, 1);

    ch1 = d(:, 2);

    ch2 = d(:, 3);

    e1 = find(diff(ch1 > mean(ch1)) ~= 0);
```

```

e2 = find(diff(ch2 > mean(ch2)) ~= 0);

if std(diff(e1)) > std(diff(e2))

    sig = ch1;

else

    sig = ch2;

end

thresh = mean(sig);

bpsig = 2 * double(sig > thresh) - 1;

[ac, lags] = xcorr(bpsig, 'normalized');

dt = t(2) - t(1);

tshifts = lags * dt;

subplot(3, 1, i);

plot(tshifts, ac, 'LineWidth', 1);

title(plottitles{i});

xlabel('Time Shift (s)');

ylabel('AC');

grid on;

ylim([-0.3 1.1]);

xlim([-1e-3 1e-3]);

end

|

%% Software Simulations

figure(2);
seq1 = get_lfsr(6, [6, 1]);

ac1 = calc_ac(seq1);
subplot(2, 1, 1);

stem(-(length(seq1)):length(seq1), ac1, 'filled', 'MarkerSize', 4);

title('Primitive (x^6 + x + 1)');

```

```
xlabel('Shift');
```

```
ylabel('AC');
```

```
grid on; axis([-63 63 -0.5 1.1]);
```

```
seq2 = get_lfsr(6, [6, 5, 4, 3, 2, 1]);
```

```
ac2 = calc_ac(seq2);
```

```
subplot(2, 1, 2);
```

```
stem(-(length(seq2)):length(seq2), ac2, 'filled', 'MarkerSize', 4);
```

```
title('Non-Primitive ( $x^6 + x^5 + x^4 + x^3 + x^2 + x + 1$ )');
```

```
xlabel('Shift');
```

```
ylabel('AC');
```

```
grid on; axis([-63 63 -0.5 1.1]);
```

```
function s = get_lfsr(stages, taps)
```

```
    state = ones(1, stages);
```

```
    L = 2^stages - 1;
```

```
    s = zeros(1, L);
```

```
    for k = 1:L
```

```
        s(k) = state(end);
```

```
        fb = mod(sum(state(taps)), 2);
```

```
        state = [fb, state(1:end-1)];
```

```
    end
```

```
end
```

```
function ac = calc_ac(s)
```

```
    N = length(s);
```

```
    bp = 2 * s - 1;
```

```
    res = zeros(1, N);
```

```
    for k = 0:N-1
```

```
        shifted = circshift(bp, k);
```

```
        res(k+1) = sum(bp .* shifted) / N;
```

end

```
p_shifts = [res, res(1)];
```

```
ac = [p_shifts(end:-1:2), p_shifts];
```

end

Observation:

- **Primitive Polynomial (x^6+x+1):** This polynomial successfully generated a maximum-length sequence with a period of $N=2^6-1=63$ bits. The discrete stem plot shows a perfect single spike of 1.0 at shifts of 0, 63, and -63, with a constant, flat minimum of $-1/63$ everywhere else.
- **Non-Primitive Polynomial ($x^6+x^5+x^4+x^3+x^2+x+1$):** This polynomial failed to generate an m-sequence. The LFSR fell into a short cycle, repeating itself every 7 bits. Consequently, its autocorrelation plot shows peaks every 7 shifts, proving that only primitive polynomials can generate maximum-length PN sequences with ideal correlation properties.

DISCUSSION

Before analyzing our results, it's helpful to define exactly what autocorrelation means in this context. Mathematically, the autocorrelation function $R_{xx}(k)$ of a sequence $x(i)$ of length N measures how similar the sequence is to a time-shifted version of itself, defined as:

$$\frac{1}{N} \sum_{i=0}^{N-1} x(i)x(i+k)$$

With this definition in mind, our hardware and software results made complete sense:

- **Periodicity:** For our 5-bit LFSR using the x^5+x^3+1 polynomial, the sequence repeated exactly every 31 clock cycles. This perfectly matched the theoretical $N=2^5-1$ rule for maximum-length sequences.
- **Maxima at the Zeroth-Shift:** When we plotted the autocorrelation (both in MATLAB and via the oscilloscope's math functions), the function hit an absolute maximum exactly at a shift of zero ($k=0$). Looking at the formula, this happens because at zero shift, every single bit perfectly aligns with itself, so the summation reaches its maximum value of 1.
- **Autocorrelation Properties:** For any shift other than zero (or multiples of 31), the sequence no longer aligned with itself. Because of the balance property of PN sequences, the product $x(i)x(i+k)$ resulted in roughly an equal number of positive and negative ones, dropping the correlation to a constant, flat minimum.

Finally, our experiment with Gold sequences highlighted an important trade-off. While m-sequences give us that beautiful, flat autocorrelation, they can have unpredictable cross-correlation with other sequences. By XORing two PN sequences to create a Gold sequence, we sacrificed a little bit of that perfect flatness to guarantee bounded, predictable cross-correlation—a crucial requirement for systems like CDMA where multiple signals share the same channel.

INFERENCES

From the experimental and simulation observations, a few key things stood out:

1. **Primitive vs Non-Primitive Polynomials:** During the MATLAB software simulation, it became crystal clear that you can't just pick any random polynomial if you want an m-sequence. A non-primitive polynomial completely failed and got stuck in a short loop (our 7-bit repeating cycle). A primitive polynomial is an absolute strict requirement to get maximum length and ideal correlation properties.
2. **The Lockout Condition:** The "lockout" state is a very real hardware issue. If the XOR feedback ever hits an all-zero state, the whole circuit essentially dies. Adding the 2:1 MUX and OR gate was a simple but highly effective way to kickstart the system back into action.
3. **Practical Application of Gold Sequences:** While m-sequences are great for basic spread spectrum applications, Gold sequences are the actual practical solution when you need multiple unique codes for different users in a network, due to their controlled cross-correlation bounds.

CONCLUSION

To wrap things up, this experiment provided a really solid hands-on understanding of how Pseudo-Noise sequences are generated and why they matter in digital communications. We successfully built a 5-stage LFSR on the breadboard, proved it was a maximal-length sequence (31 bits), and implemented a hardware fix for the all-zero lockout problem. Furthermore, we managed to generate a 127-bit Gold sequence by combining two 7-stage LFSRs. Finally, by comparing the continuous-time hardware data against discrete MATLAB simulations, we verified that primitive polynomials are the only way to get those ideal, sharp autocorrelation peaks needed for things like signal synchronization and ranging.

APPENDIX:

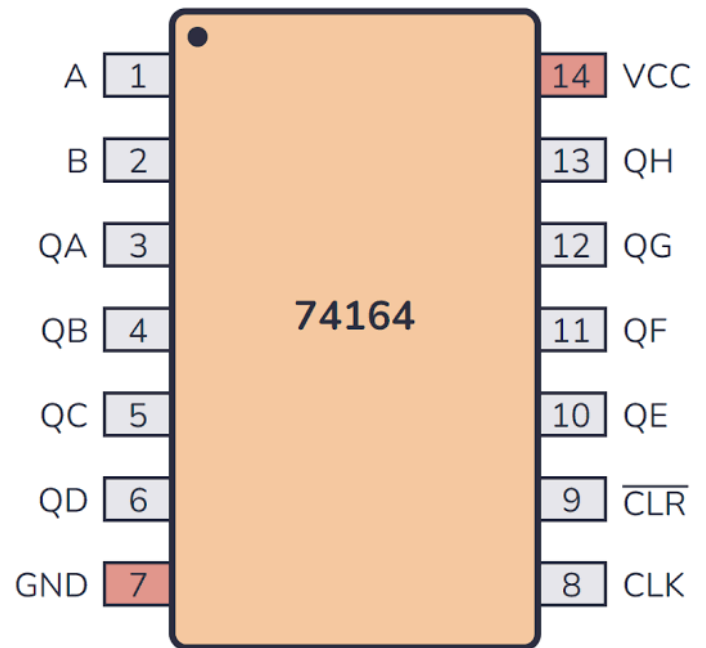
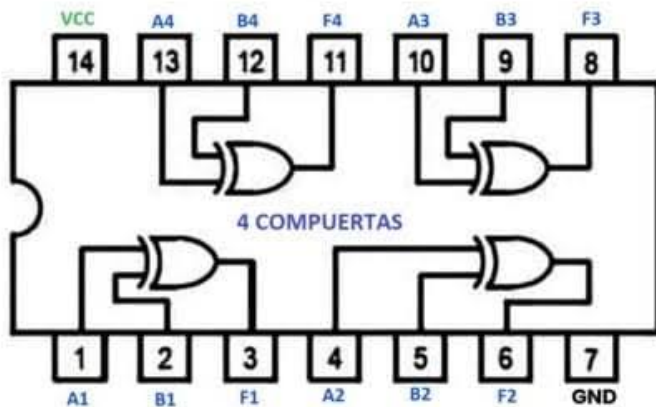
Appendix A: List of Primitive Polynomials

Degree	Primitive Polynomials
2	[2 1 0]
3	[3, 1, 0], [3, 2, 0]
4	[4, 1, 0], [4, 3, 0],
5	[5, 2, 0], [5, 3, 0], [5, 3, 2, 1, 0], [5, 4, 2, 1, 0], [5, 4, 3, 1, 0], [5, 4, 3, 2, 0]
6	[6, 1, 0], [6, 4, 3, 1, 0], [6, 5, 0], [6, 5, 2, 1, 0], [6, 5, 3, 2, 0], [6, 5, 4, 1, 0].
7	[7, 1, 0], [7, 3, 0], [7, 3, 2, 1, 0], [7, 4, 0], [7, 4, 3, 2, 0], [7, 5, 2, 1, 0]
7	[7, 5, 3, 1, 0], [7, 5, 4, 3, 0], [7, 5, 4, 3, 2, 1, 0], [7, 6, 0], [7, 6, 3, 1, 0]
7	[7, 6, 4, 1, 0], [7, 6, 4, 2, 0], [7, 6, 5, 2, 0], [7, 6, 5, 3, 2, 1, 0], [7, 6, 5, 4, 0]
7	[7, 6, 5, 4, 2, 1, 0], [7, 6, 5, 4, 3, 2, 0]
8	[8, 4, 3, 2, 0], [8, 5, 3, 1, 0], [8, 5, 3, 2, 0], [8, 6, 3, 2, 0], [8, 6, 4, 3, 2, 1, 0],
8	[8, 6, 5, 1, 0], [8, 6, 5, 2, 0], [8, 6, 5, 3, 0], [8, 6, 5, 4, 0], [8, 7, 2, 1, 0],
8	[8, 7, 3, 2, 0], [8, 7, 5, 3, 0], [8, 7, 6, 1, 0], [8, 7, 6, 3, 2, 1, 0],
8	[8, 7, 6, 5, 2, 1, 0], [8, 7, 6, 5, 4, 2, 0]

Appendix B: IC Pin Diagrams

74LS86

XOR



REFERENCES:

- [1] D. Florescu and A. Bhandari, "Time encoding via unlimited sampling: Theory, algorithms and hardware validation," IEEE Transactions on Signal Processing, vol. 70, pp. 4912–4924, 2022.
- [2] S. W. Golomb, Shift Register Sequences. San Francisco, CA, USA: Holden-Day, 1967.
- [3] J. G. Proakis and M. Salehi, Digital Communications, 5th ed. New York, NY, USA: McGraw-Hill, 2008.
- [4] B. Sklar, Digital Communications: Fundamentals and Applications, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2001.
- [5] D. V. Sarwate and M. B. Pursley, "Crosscorrelation properties of pseudorandom and related sequences," Proceedings of the IEEE, vol. 68, no. 5, pp. 593–619, May 1980.